COMP0008: Computer Architecture & Concurrency – CW2

Dear students,

The questions below will not be marked and they are for you to test your understanding.

We will publish solutions in 2-3 weeks that will allow you to check your answers.

If any of the questions is not clear, please post a question in the forum or attend the drop-in sessions.

COMP0008 staff.

Part 1: Instruction encoding:

Without using MARS:

1. write the 32-bit binary MIPS representation of $\text{mult}\ \$13, \$15$.
2. Disassemble and understand the following binary code:
   0x210affff
   0x010a4824
   0x0009582b
   - Write the three instructions in assembly.
   - ***(Challenging!)*** For which values of $8, is $11 going to be 1 after these instructions are executed?

   Note: One of these instructions is an *I*-type instruction, whose format will be discussed in week 3's lecture. Try inferring the structure from the MIPS reference card!

Part 2: The Collatz conjucture:

The Collatz conjucture is that any Collatz sequence, defined by an input integer a_0, always reaches the value 1 after a finite number of steps.

Starting with the number a_0, the sequence is defined as $a_{n+1} = \begin{cases} 3a_n + 1 & \text{if } (a_n \bmod 2) = 1 \\ a_n/2 & otherwise \end{cases}$.

For example, if $a_0 = 7$, then the sequence is $7,22,11,34,17,52,26,13,40,20,10,5,16,8,4,2,1,4,2,1, \ldots$

It was recently (July 2021) announced that whoever solves the problem would win a prize larger than $1M.

In this CW, we will (probably) not solve the conjecture.

Instead, you'll be writing a MIPS32 assembly program that will print the sequence (for up to 500 elements) of the sequence, given a user input a_0.

Specifically, write a code that takes an integer as input and prints the sequence numbers until the first 1.

You may assume that none of the numbers exceed $2^{31} - 1$ and can thus be represented using 32-bit integers.

Remember: when programming in MIPS, the reference card (link on the website) is your friend!

Part 3: Implementing complex pseudo instructions

MARS, which you will be using in this CW, has an assembler that replaces pseudo-instructions with (one or more) hardware supported instructions.

On a different topic, out-of-order-execution (OOOE) CPUs (more on that later in the module!) suffer great performance hit when a program contains many branches.

This means that branch-less programming is often essential for writing performant code.

In this assignment, you'll design an implementation of two new "pseudo instructions" that are not currently supported on MARS:

- min $t0, $t1, $t2 (set $t0 = min($t1,$t2)).
- med $t0, $t1, $t2 (set $t0 to the median of $t0,$t1,$t2).

As this is not the focus of the module, we will not actually modify MARS's code. Instead you will:

1. Write a code that prints out the minimum of two registers $t1,$t2 (which will be set at the beginning of the code). Your code must be branchless!
2. ***(Challenging!)*** Write a code that prints out the median of three registers $t0,$t1,$t2 (which will be set at the beginning of the code). Your code must be branchless!

Clarification: In general, as we will discuss in week 4's lecture, implementation of pseudo instruction may not modify the content of registers other than the goal register ($t0 in these examples) and $1.

While this is doable for the min operation, for implementing the median you may use all temporary registers ($t0-$t7) as if it was a function call.

Clarification: only the code of the min/med operations should be branchless. It's okay to make a syscall for outputting the result.

A code template (that you can, but not must, use as a starting point) appears in the next page.

Note: $t0-$t7 are aliases to $8-$15, where the *t* stands for temporary. We will discuss the different register roles in future weeks.

```
.data
msg: .asciiz "\nThe median is:\n"
.text
main:

li $v0, 4 #syscall with v0=4 --> print string
la $a0, msg #call parameter --> pointer to the message
syscall

li $t0, 31321 #Example values, should work for any combination
li $t1, 2179    #A pseudo instruction that sets $t1 = 2179
li $t2, 1178914

### Actual branchless code of computing $t0 <- median($t0,$t1,$t2)

### End of your code

move $a0, $t0  #Set $a0 <- $t0
li $v0, 1 #syscall 1 -- prints the integer $a0
syscall
```